Programmation 3D
Shaders 2

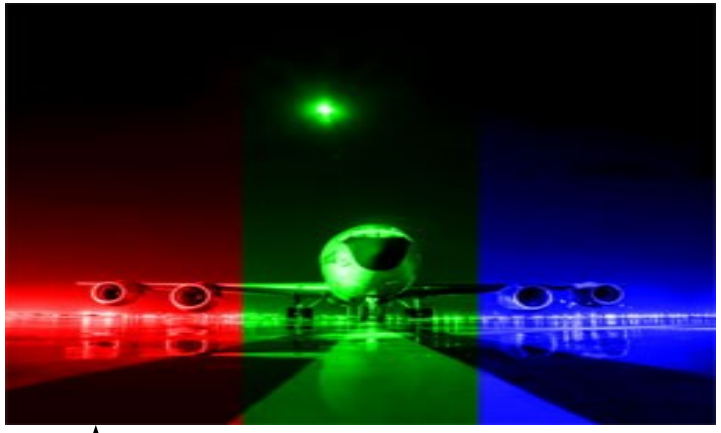
Objectif

L'objectif de ce cours est d'explorer certaines techniques courantes utilisées lors de la création de shaders.

Filtres et Blend

Filtre couleur

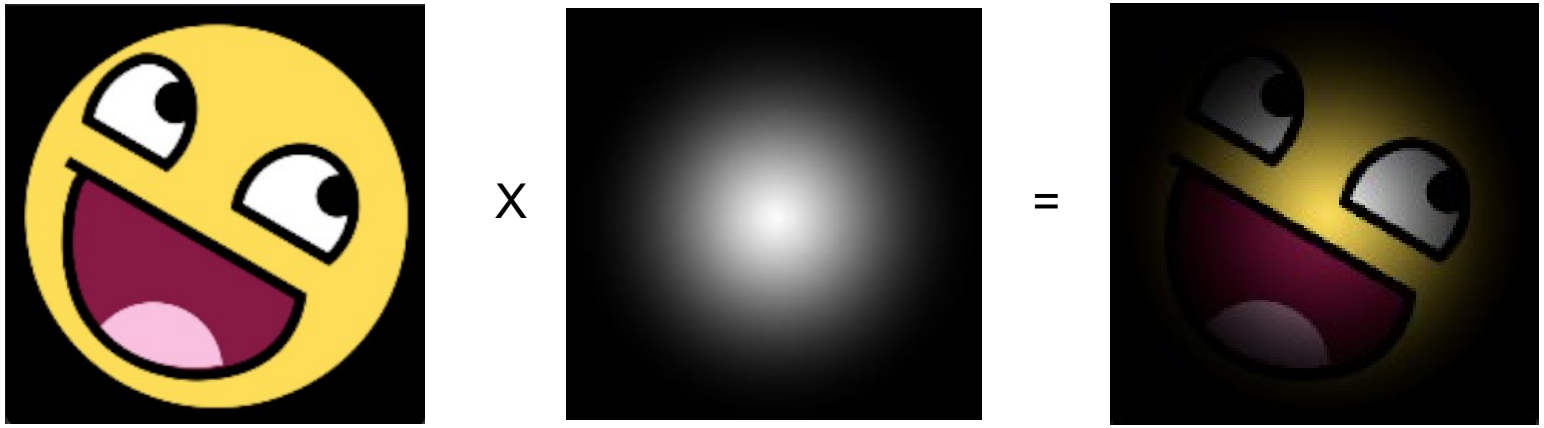
Valeurs R, G et B de la couleur rendue pour chaque pixel modifié pour créer un filtre couleur.



```
color = float4(color.r, 0, 0, 1);
```

Textures Blending

Deux textures échantillonnées par pixel. Valeurs échantillonnées "mélangées" :




```
   float4 col = tex2D(_MainTex, uv);
float4 colBlend = tex2D(_TexBlend, uv);
   col.rgb = col.rgb * colBlend.rgb;
```

Clipping / Fading

Une texture échantillonnée par pixel peut permettre de déterminer la valeur alpha de la couleur rendue pour ce pixel :

+

=

```
float4 col = tex2D(_MainTex, uv);
float4 colMask = tex2D(_TexBlend, uv);
col.a = step(0.5, colMask.r);
```

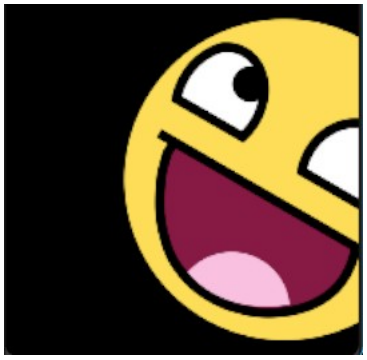
Echantillonage

UV Offset

Valeur de translation UV servant à échantillonner une texture modifiée pour décaler la texture rendue.

+ UV Offset

=



```
float2 uvOffset = float2(_Time.y, 0);
float4 col = tex2D(_MainTex, uv + uvOffset);
```

UV Displacement Map

Une texture échantillonnée par pixel permettant de déterminer un offset UV pour une autre texture échantillonnée sur ce même pixel.

```
float offset = lerp(-1, 1, tex2D(_TexBlend, uv).r) * power;
float2 uvOffset = float2(0, offset);
float4 col = tex2D(_MainTex, uv + uvOffset);
```

SDF

SDF : Définition

SDF : Signed Distance Function

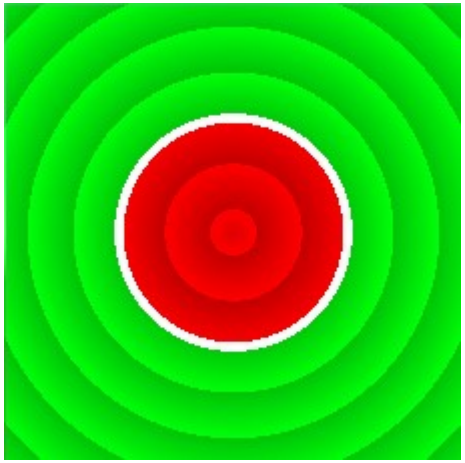
Permet de représenter une forme géométrique sous forme de formule mathématique de distance d'un point de l'espace à cette forme.

La distance est négative à l'intérieur de la forme et positive à l'extérieur de la forme.

SDF : Circle

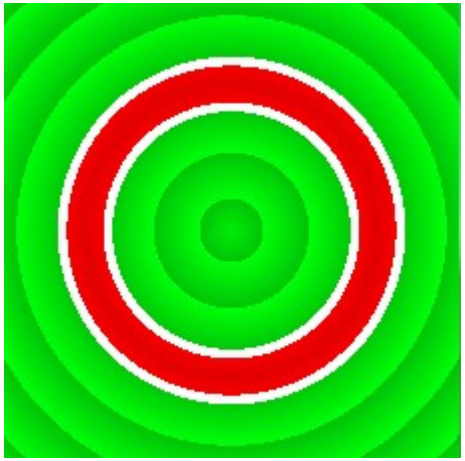
- Cercle (r) :

```
d = length(p) - r
```



- Anneau (r, w):

```
d = abs(length(p) - r) - w * 0.5
```



SDF : Box

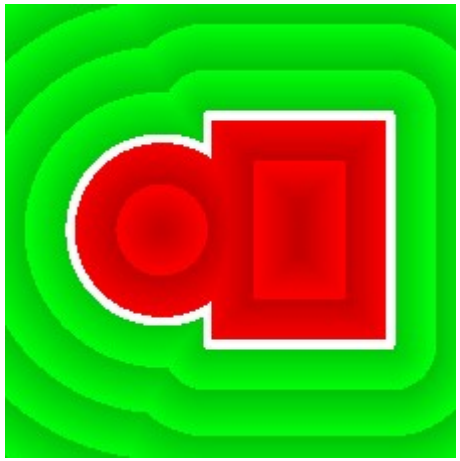
- Box (w, h):

```
float2 centerToCorner = float2(w * 0.5, h * 0.5);
float2 offset = abs(p) – centerToCorner;
d = length(max(offset, 0)) + min(max(offset.x, offset.y), 0);
```

SDF : Combine

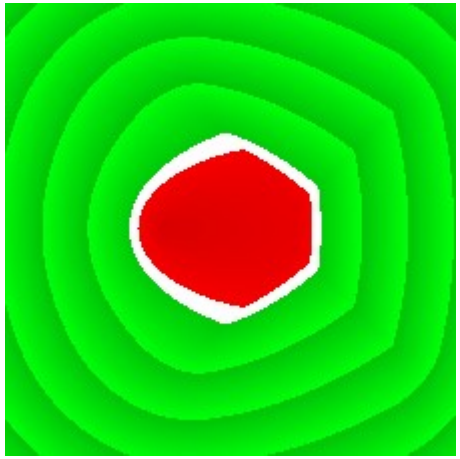
- Merge (d1, d2):

```
d = min(d1, d2)
```



- Lerp (d1, d2, t):

```
d = lerp(d1, d2, t)
```



SDF : Plus loins

Autres formes :

- Triangles, arc de cercles, boites orientées, polygones, ...
- Etoiles, lunes, coeurs, Ellipses, ...
- Formes 3D
- ...

Autres combinaisons :

- Additions, soustractions, fusions arrondies, ...

Displacement Shader

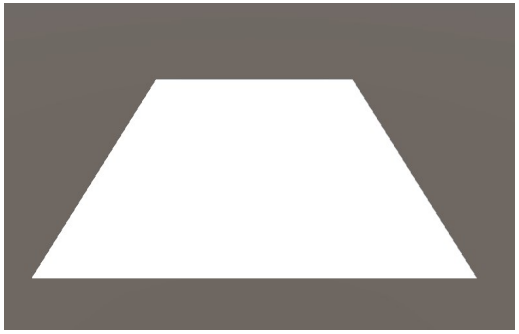
Displacement Shader

En déplaçant tous les vertices d'un mesh dans le sens de leur normale, on peut faire gonfler un mesh (utile pour réaliser un outline) :

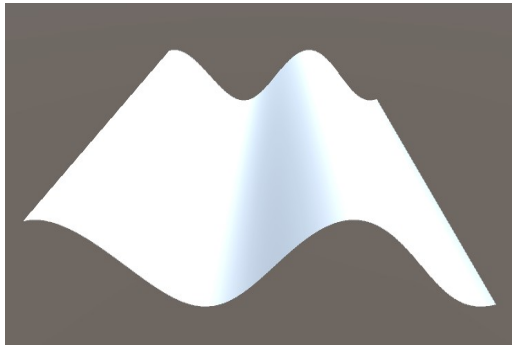
```
o.position += v.normal.xyz * power;
```

Displacement Shader

En déplaçant tous les vertices d'un mesh en fonction de leur position, soit avec une fonction, soit avec une texture, on peut obtenir des effets de propagation d'onde.



+ Sinus =

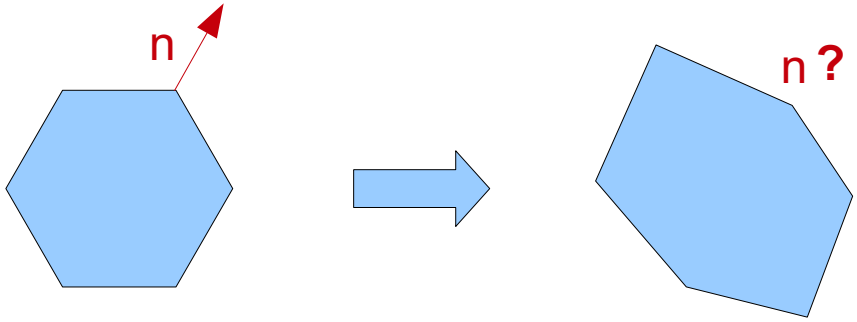


```
o.position += v.normal.xyz * scale * sin(v.position.x *
frequency);
```

Rappel : Vertex Shader

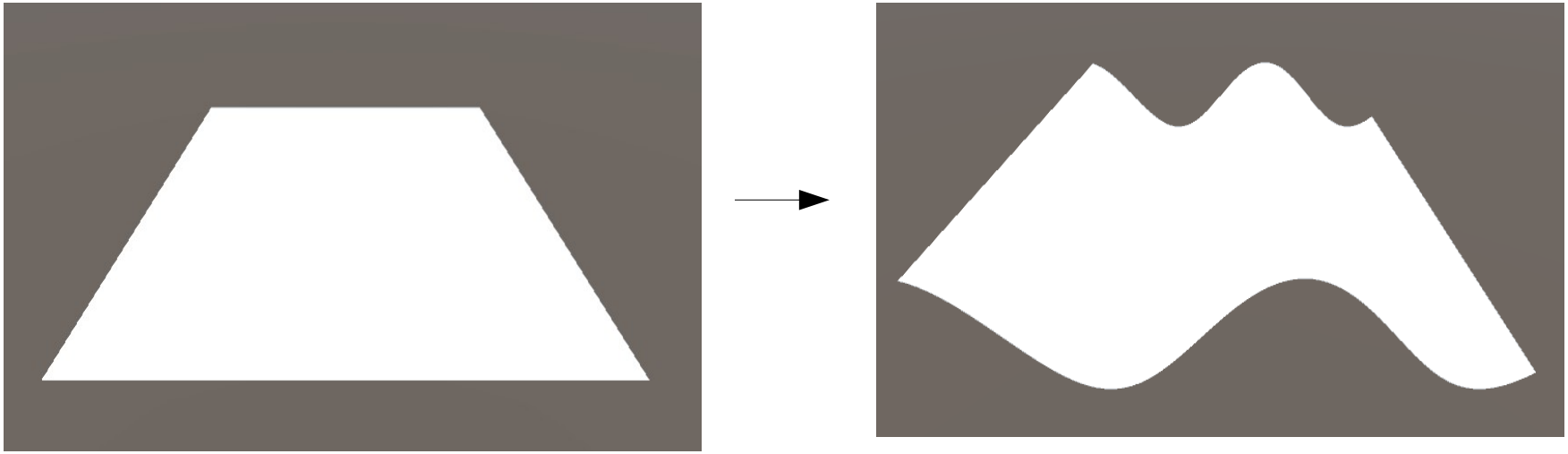
Un vertex traité par la fonction de vertex shader ne connait pas ses vertices avoisinants !

De ce fait, il est difficile de travailler sur la position des vertices en gardant des normales de vertex cohérentes.



Displacement & Normales

Exemple de displacement shader sans réparation des normales :

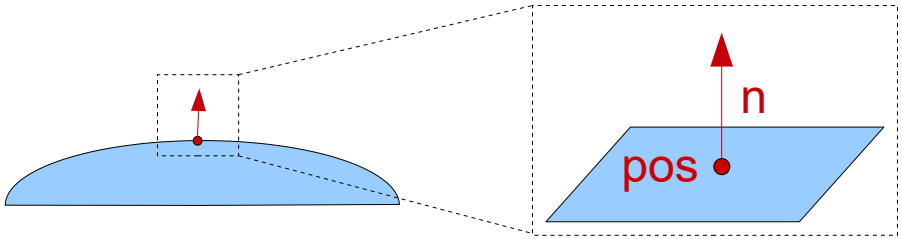


Displacement & Normales

Il est possible de réparer la normale d'un vertex déplacé si ce déplacement est calculé par une fonction continue dépendant de sa position.

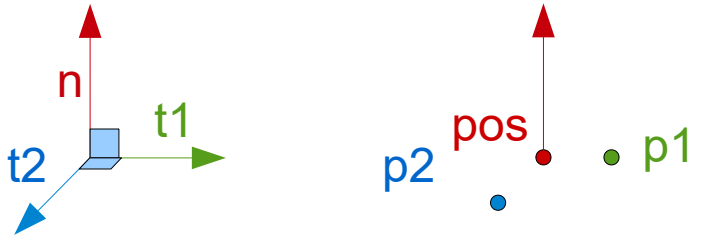
```
o.pos = moveVertex(v.pos);
```

Pour réparer la normale, on approxime que la surface du mesh est plane localement à très petite échelle.



Displacement & Normales

Une tangente à la normale et sa bitangente peuvent être calculées à partir de la normale (produit vectoriel) :



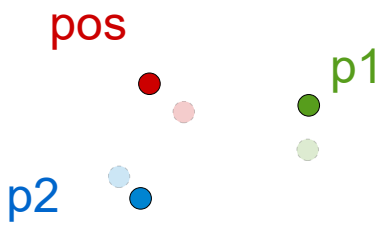
En utilisant ces tangentes, on détermine deux points du plan très proche du vertex déplacé :

```
float3 p1 = v.pos + t1 * Epsilon;
float3 p2 = v.pos + t2 * Epsilon;
```

Displacement & Normales

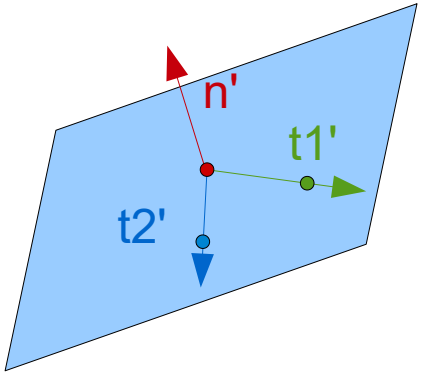
On applique la fonction de déplacement sur chaque point :

```
o.pos = moveVertex(v.pos);
float3 p1 = moveVertex(v.pos + t1 * Epsilon);
float3 p2 = moveVertex(v.pos + t2 * Epsilon);
```



Displacement & Normales

On peut alors retrouver deux nouvelles tangentes au plan à partir des 3 points déplacés, donc une nouvelle normale (produit vectoriel).



C'est la nouvelle normale pour notre vertex !

Effet Fresnel

Effet Fresnel

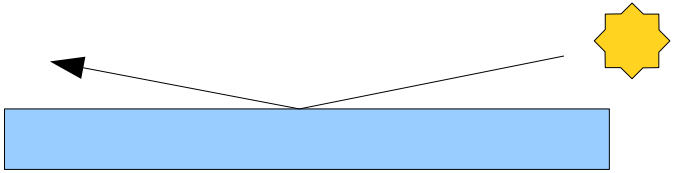
Plus de réflexions

Moins de réflexions

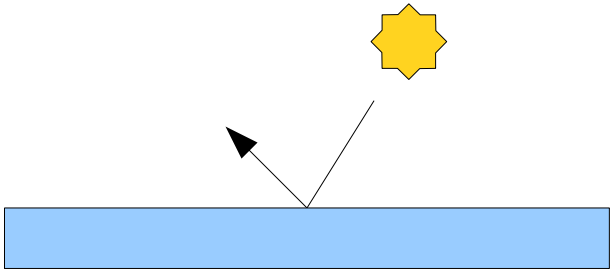
Effet Fresnel

Plus l'angle de rebond de la lumière sur une surface est obtus, plus la réflexion est dure. Plus il est aigu, plus la réflexion est faible :

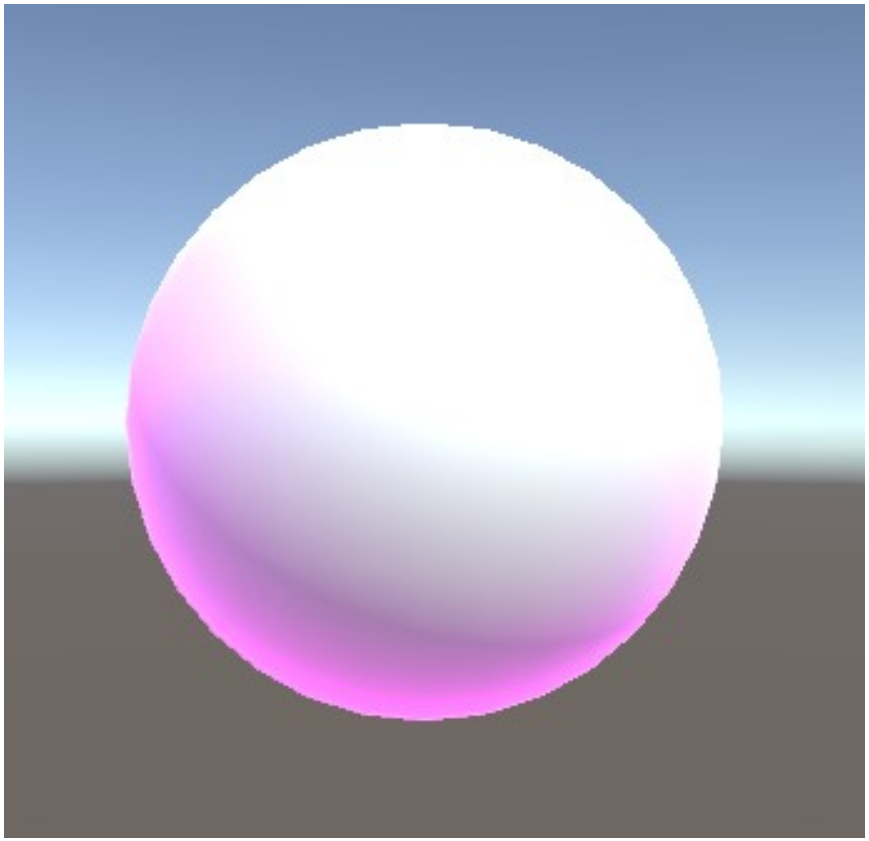
Plus de réflexions



Moins de réflexions



Effet Fresnel



Effet Fresnel

En utilisant le vecteur directeur de la caméra et la normale à la surface rendue, l'effet Fresnel se calcule (approximation) avec la formule suivante :

```
float GetFresnelEffect(float3 normal, float3 viewDir, float power) {
    float dotResult = dot(normalize(normal), normalize(viewDir));
return pow((1.0 – saturate(dotResult)), power);
}
// saturate <=> clamp01

...
emissive += fresnelColor * GetFresnelEffect(...));
// OU : col = lerp(col, fresnelColor, GetFresnelEffect(...));
```